

The logo for Oracle Academy. The word "ORACLE" is in a bold, orange, sans-serif font. Below it, the word "Academy" is in a smaller, dark gray, sans-serif font. The entire logo is centered on a light gray background, which is framed by dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Programming with PL/SQL

14-1

Introduction to Dependencies

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

- This lesson covers the following objectives:
 - Describe the implications of procedural dependencies
 - Contrast dependent objects and referenced objects
 - View dependency information in the data dictionary
 - Use the UTLDTREE script to create the objects required to display dependencies
 - Use the IDEPTREE and DEPTREE views to display dependencies
 - Describe when automatic recompilation occurs
 - List how to minimize dependency failures

Purpose

- A PL/SQL subprogram can execute correctly only if the objects that it references exist and are valid
- These objects can be tables, views, other PL/SQL subprograms, and other kinds of database objects
- So what happens if a referenced object is altered or dropped?
- This lesson introduces you to object dependencies and implicit and explicit recompilation of invalid objects

Understanding Dependencies

- Some objects reference other objects as part of their definitions
- For example, a stored procedure could contain a SELECT statement that selects columns from a table
- For this reason, the stored procedure is called a dependent object, whereas the table is called a referenced object



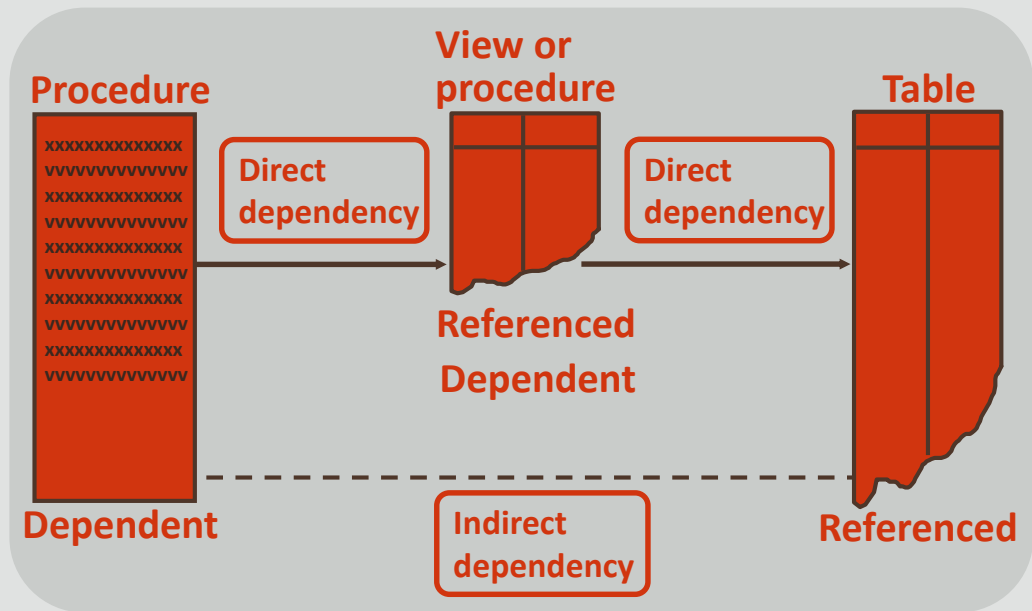
Dependency Issues

- If you alter the definition of a referenced object, dependent objects might not continue to work properly
- For example, if the table definition is changed, the procedure might not continue to work without error
- The Oracle server automatically records dependencies among objects
- To manage dependencies, all schema objects have a status (valid or invalid) that is recorded in the Data Dictionary, and you can view the status in the USER_OBJECTS Data Dictionary view

Dependency Issues

- A procedure or function can directly or indirectly (through an intermediate view, procedure, function, or packaged procedure or function) reference the following objects:
 - Tables
 - Views
 - Sequences
 - Procedures
 - Functions
 - Packaged procedures or functions

Dependencies



Dependencies Summarized

- Note : Some object types (sometimes the same object) can be both dependent and referenced

Dependent Objects

Table
View
Database trigger
Procedure
Function
Package body
Package specification
User-defined object
and collection types

Referenced Objects

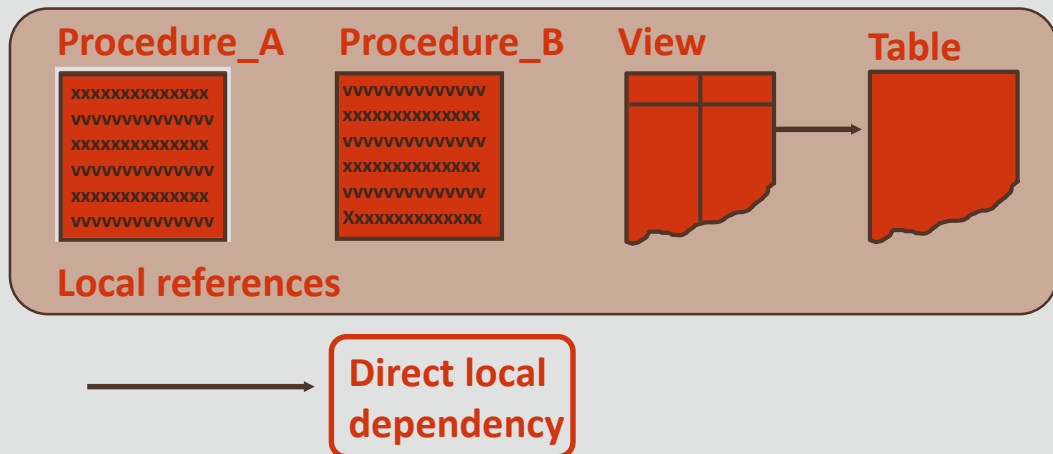
Function
Package specification
Procedure
Sequence
Synonym
Table
View
User-defined object
and collection types



Local Dependencies

- In the case of local dependencies, the objects are on the same node in the same database
- The Oracle server automatically manages all local dependencies, using the database's internal "depends-on" table
- When a referenced object is modified, the dependent objects are invalidated
- The next time an invalidated object is called, the Oracle server automatically tries to recompile it
- If the referenced object is in a different database, the dependency is called a remote dependency
- Because the two databases have separate data dictionaries, some of the features described in this lesson work differently for remote dependencies

Local Dependencies



ORACLE
Academy

PLSQL 14-1
Introduction to Dependencies

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

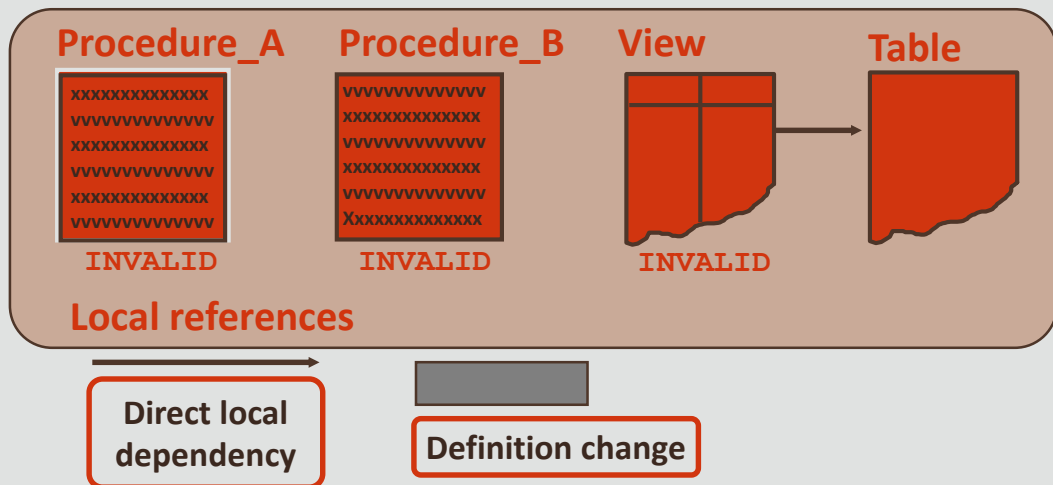
11

Assume that the structure of the table on which a view is based is modified. When you describe the view by using the DESCRIBE command, you get an error message that states that the object is invalid to describe. The view is invalid because the structure of its base table is changed. If you query the view, the view is recompiled automatically and you can see the result (if it is successfully recompiled).

Although this example discusses automatic recompilation of a view, the same mechanisms apply to other dependent objects such as subprograms.

Local Dependencies

- The Oracle server implicitly attempts to recompile any INVALID object when the object is next called



A Scenario of Local Dependencies

ADD_EMP procedure

```
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
```

EMP_VW view

EMPLOYEE_ID	LAST_NAME	FIRST_NAME	EMAIL	DEPARTMENT_ID
100	King	Steven	SKING	90
101	Kochhar	Neena	NKOCHHAR	90
102	De Haan	Lex	LDEHAAN	90
200	Whalen	Jennifer	JWHALEN	10

QUERY_EMP procedure

```
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxx
```

EMPLOYEES table

EMPLOYEE_ID	LAST_NAME	FIRST_NAME	EMAIL	PHONE_NUMBER
100	King	Steven	SKING	515.123.4567
101	Kochhar	Neena	NKOCHHAR	515.123.4568
102	De Haan	Lex	LDEHAAN	515.123.4569
200	Whalen	Jennifer	JWHALEN	515.123.4444

ORACLE
Academy

PLSQL 14-1
Introduction to Dependencies

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

13

The QUERY_EMP procedure directly references the EMPLOYEES table. The ADD_EMP procedure updates the EMPLOYEES table indirectly by using the EMP_VW simple view.

A Scenario of Local Dependencies

- In each of the following cases, will the ADD_EMP procedure be invalidated, and will it successfully recompile when next invoked?
 - 1. The internal logic of the QUERY_EMP procedure is modified
 - 2. A new column is added to the EMPLOYEES table
 - 3. The EMP_VW view is dropped

Answers:

1. ADD_EMP will NOT be invalidated because it is not directly or indirectly dependent on QUERY_EMP.
2. ADD_EMP will be invalidated because it is indirectly dependent on EMPLOYEES. When ADD_EMP is next invoked, it will recompile successfully (and so will EMP_VW).
3. ADD_EMP will be invalidated because it is directly dependent on EMP_VW. When next invoked, it will not recompile successfully because its referenced object (EMP_VW) no longer exists.

Displaying Direct Dependencies

- Direct dependencies can be viewed through the `USER_DEPENDENCIES` and `ALL_DEPENDENCIES` views

```
SELECT name, type, referenced_name, referenced_type
FROM user_dependencies
WHERE referenced_name IN ( 'EMPLOYEES', 'EMP_VW' );
```

- You can also view direct dependencies in Application Express: SQL Workshop -> Object Browser -> choose an object, then click the Dependencies tab

Determine which database objects to recompile manually by displaying direct dependencies from the `USER_DEPENDENCIES` data dictionary view.

Examine the `ALL_DEPENDENCIES` view, which contains the additional column `OWNER`, which references the owner of the object.

Note: the `*_DEPENDENCIES` views do not show indirect dependencies. For these we need to run the `utldtreel.sql` script as shown in the next slide.

Displaying Direct and Indirect Dependencies

- Download and unzip the utldtree.sql script from Section 14, Lesson 1 resources in Oracle iLearning
- Upload and run the script in your APEX schema (SQL Workshop -> SQL Scripts)
- This script creates four objects:
 - A table deptree_temptab to hold dependency data
 - A procedure deptree_fill to populate the table
 - Two views deptree and ideptree to select and format dependency data from the populated table

See the APEX_Student_Guide in Section 0 if you need assistance with running the `utldtree.sql` script.

Displaying Direct and Indirect Dependencies

- For each object whose dependencies you want to see:
- 1. Execute the DEPTREE_FILL procedure

```
BEGIN deptree_fill('TABLE','SCOTT','EMPLOYEES'); END;
```

- This populates the DEPTREE_TEMPTAB table with information for a particular referenced object
- The three parameters for this procedure are:
 - object_type: Type of the referenced object
 - object_owner: Schema of the referenced object
 - object_name: Name of the referenced object

You can verify that the utldtree.sql script ran if you DESCRIBE its structure:

DESCRIBE deptree temptab.

Displaying Direct and Indirect Dependencies

- 2. Display the dependency data using the DEPTREE view

```
SELECT nested_level, type, name
FROM   deptree
ORDER BY seq#;
```

- In this example, ADD_EMP is directly dependent on EMP_VW, which in turn is directly dependent on EMPLOYEES (look at the NESTED-LEVEL column)

NESTED_LEVEL	TYPE	NAME
0	TABLE	EMPLOYEES
1	VIEW	EMP_VW
2	PROCEDURE	ADD_EMP
1	PROCEDURE	QUERY_EMP

Display an indented representation of the same information by querying the IDEPTREE view, which consists of a single column named DEPENDENCIES.

For example:

```
SELECT * FROM ideptree;
```

provides a single column of indented output of the dependencies in a hierarchical structure.

Another Scenario of Local Dependencies

**REDUCE_SAL
procedure**

RAISE_SAL procedure

EMPLOYEES table

EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
100	King	AD_PRES	24000
101	Kochhar	AD_VP	17000
102	De Haan	AD_VP	17000
200	Whalen	AD_ASST	4400
205	Higgins	AC_MGR	12000

XXXXXXXXXXXXXXXXXXXXX
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
XXXXXXXXXXXXXXXXXXXXX
VVVVVVVVVVVVVVVVVVVV



XXXXXXXXXXXXXXXXXXXXX
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
VVVVVVVVVVVVVVVVVVVV
XXXXXXXXXXXXXXXXXXXXX
VVVVVVVVVVVVVVVVVVVV

Another Scenario of Local Dependencies

- Suppose that the RAISE_SAL procedure (from the previous slide) updates the EMPLOYEES table directly, and that the REDUCE_SAL procedure updates the EMPLOYEES table indirectly by way of RAISE_SAL
- In each of the following cases, will the REDUCE_SAL procedure successfully recompile?
 - 1. The internal logic of the RAISE_SAL procedure is modified
 - 2. One of the formal parameters to the RAISE_SAL procedure is eliminated

Answers:

1. REDUCE_SAL will successfully recompile because its actual parameters have not changed, therefore its invocation from REDUCE_SAL is still valid.
2. REDUCE_SAL will not successfully recompile because one of its actual parameters in its call to RAISE_SAL no longer exists.

A Third Scenario of Local Naming Dependencies

QUERY_EMP procedure

```
xxxxxxxxxxxxxxxxxxxxx
vvvvvvvvvvvvvvvvvvvv
vvvvvvvvvvvvvvvvvvvv
vvvvvvvvvvvvvvvvvvvv
vvvvvvvvvvvvvvvvvvvv
vvvvvvvvvvvvvvvvvvvv
xxxxxxxxxxxxxxxxxxxxx
vvvvvvvvvvvvvvvvvvvv
```



EMPLOYEES public synonym

EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
100	King	AD_PRES	24000
101	Kochhar	AD_VP	17000
102	De Haan	AD_VP	17000
200	Whalen	AD_ASST	11000

EMPLOYEES table

EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
100	King	AD_PRES	24000
101	Kochhar	AD_VP	17000
102	De Haan	AD_VP	17000
200	Whalen	AD_ASST	11000

Be aware of the subtle case in which the creation of a table, view, or synonym may unexpectedly invalidate a dependent object because it interferes with the Oracle server hierarchy for resolving name references.

Predict the effect that the name of a new object has upon a dependent procedure.

Suppose that your QUERY_EMP procedure originally referenced a public synonym called EMPLOYEES. However, you have just created a new table called EMPLOYEES within your own schema.

Questions:

Does this change invalidate the procedure?

Answer: Yes

Which of the two EMPLOYEES objects does QUERY_EMP reference when the procedure recompiles?

Answer: your own EMPLOYEES table.

Now suppose that you drop your private EMPLOYEES table. Does this invalidate the procedure?

Answer: Yes.

Now what happens when the procedure recompiles?

Answer: It will recompile successfully and reference the public synonym.

You can track security dependencies in the USER_TAB_PRIVS data dictionary view.

A Third Scenario of Local Naming Dependencies

- Be aware of the subtle case in which the creation of a table, view, or synonym may unexpectedly invalidate a dependent object because it interferes with the Oracle server hierarchy for resolving name references
- Predict the effect that the name of a new object has upon a dependent procedure
- Suppose that your QUERY_EMP procedure (from the previous slide) originally referenced a public synonym called EMPLOYEES
- However, you have just created a new table called EMPLOYEES within your own schema

A Third Scenario of Local Naming Dependencies

- Does this change invalidate the procedure?
- Which of the two EMPLOYEES objects does QUERY_EMP reference when the procedure recompiles?
- Now suppose that you drop your private EMPLOYEES table
- Does this invalidate the procedure?
- Now what happens when the procedure recompiles?
- You can track security dependencies in the USER_TAB_PRIVS data dictionary view

Answers:

#1 Yes

#2 Your own EMPLOYEES table.

#3 Yes.

#4 It will recompile successfully and reference the public synonym.

Recompiling a PL/SQL Program Unit

- Recompilation is either:
 - Handled automatically through implicit run-time recompilation
- OR
 - Handled through explicit recompilation with the ALTER statement:

```
ALTER PROCEDURE [SCHEMA.]procedure_name COMPILE;
```

```
ALTER FUNCTION [SCHEMA.]function_name COMPILE;
```

```
ALTER PACKAGE [SCHEMA.]package_name  
COMPILE [PACKAGE | SPECIFICATION | BODY];
```

```
ALTER TRIGGER trigger_name [COMPILE[DEBUG]];
```

ORACLE
Academy

PLSQL 14-1
Introduction to Dependencies

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

24

The DEBUG option instructs the PL/SQL compiler to generate and store the code for use by the PL/SQL debugger.

Recompiling a PL/SQL Program Unit

- If the recompilation is successful, the object becomes valid
- If not, the Oracle server returns an error and the object remains invalid
- When you recompile a PL/SQL object, the Oracle server first recompiles any invalid object on which it depends
- See notes for detailed information on object types

Procedure: Any local objects that depend on a procedure (such as procedures that call the recompiled procedure or package bodies that define the procedures that call the recompiled procedure) are also invalidated, even if they were previously valid. This is because the recompiled procedure could have been edited before recompilation.

Packages: The COMPILE PACKAGE option recompiles both the package specification and the body, regardless of whether it is invalid. The COMPILE SPECIFICATION option recompiles the package specification. Recompiling a package specification invalidates any local objects that depend on the specification, such as subprograms that use the package. Note that the body of a package also depends on its specification. The COMPILE BODY option recompiles only the package body, and does not invalidate objects dependent on the specification.

Triggers: Explicit recompilation eliminates the need for implicit run-time recompilation and prevents associated run-time compilation errors and performance overhead.

Unsuccessful Recompilation

- Recompiling dependent procedures and functions is unsuccessful when:
 - The referenced object is dropped or renamed
 - The data type of the referenced column is changed
 - The referenced column is dropped
 - A referenced view is replaced by a view with different columns
 - The parameter list of a referenced procedure is modified

Successful Recompilation

- Recompiling dependent procedures and functions is successful if:
 - New columns are added to a referenced table
 - All INSERT statements include a column list
 - No new column is defined as NOT NULL
 - The data type of referenced columns has not changed
 - A private table is dropped, but a public table that has the same name and structure exists
 - The PL/SQL body of a referenced procedure has been modified and recompiled successfully



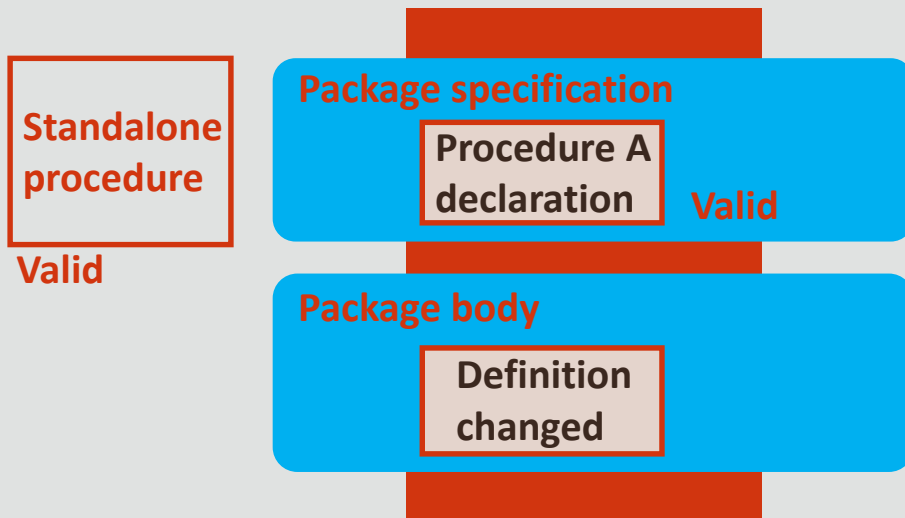
When a private table is referenced by a dependent procedure and the private table is dropped, the status of the dependent procedure becomes invalid. When the procedure is recompiled (either explicitly or implicitly) and a public table exists, the procedure can recompile successfully but is now dependent on the public table. The recompilation is successful only if the public table contains the columns that the procedure requires; otherwise, the status of the procedure remains invalid.

Recompilation of Procedures

- Minimize dependency failures by:
 - Declaring records with the %ROWTYPE attribute
 - Declaring variables with the %TYPE attribute
 - Querying with the SELECT * notation
 - Including a column list with INSERT statements



Packages and Dependencies



Packages and Dependencies

- You can simplify dependency management and avoid unnecessary invalidations with packages when referencing a package procedure or function from a stand-alone procedure or function
 - If the package body changes and the package specification does not change, then the stand-alone procedure that references a package construct remains valid
 - If the package specification changes, then the outside procedure referencing a package construct is invalidated, as is the package body

Packages and Dependencies

- If a stand-alone procedure that is referenced within the package changes, then the entire package body is invalidated, but the package specification remains valid

Therefore, it is recommended that you bring the procedure into the package

Standalone procedure

Definition changed

Package specification

Procedure A declaration

Valid

Package body

Procedure A declaration

Invalid

Summary

- In this lesson, you should have learned how to:
 - Describe the implications of procedural dependencies
 - Contrast dependent objects and referenced objects
 - View dependency information in the dictionary views
 - Use the UTLDTREE script to create the objects required to display dependencies
 - Use the IDEPTREE and DEPTREE views to display dependencies
 - Describe when automatic recompilation occurs
 - List how to minimize dependency failures

The Oracle Academy logo is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font, with the letters "R", "A", and "C" having a unique, slightly slanted design. Below "ORACLE" is the word "Academy" in a dark gray, sans-serif font. The entire logo is framed by two horizontal dark gray bars, one at the top and one at the bottom.

ORACLE

Academy